AI ASSISTANT CODING ASSIGNMENT -13.5

NAME : MOGILI KEERTHI

ROLL NO : 2303A51427

BATCH NO : 21

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

• Identify code smells and inefficiencies in legacy Python scripts.

• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

• Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1

def calculate_interest(amount):

return amount * rate

print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

Code:

```
'''Write a code to remove unnecessary global variables.
Identify any global variables used across functions and replace them by
passing values through function parameters or a configuration object.
Improve modularity and testability.
Legacy Code:
rate = 0.1
def calculate_interest(amount):
    return amount * rate
print(calculate_interest(1000))
Expected:
- No global variables
- "rate" should be passed as a function parameter or included in a config structure
- Code must be clean, modular, and testable'''
def calculate_interest(amount, rate):
    return amount * rate
# Example usage
if __name__ == "__main__":
    amount = float(input("Enter the amount: "))
    rate = float(input("Enter the interest rate: "))
    print(calculate_interest(amount, rate))
```

Output:

```
PS C:\Users\keert\auth-system> python -u "c:\Users\keert\O
Enter the amount: 1000
Enter the interest rate: 0.1
100.0
PS C:\Users\keert\auth-system>
```

Observation:

The legacy code relies on a global variable (rate), which reduces modularity and makes testing difficult. By passing rate as a function parameter, the dependency becomes explicit and the function becomes more reusable. The refactored version removes hidden state and improves clarity. This approach also enables unit testing with different rates without modifying global state.Overall, the code becomes cleaner, modular, and easier to maintain.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Code:

```python
'''
write a legacy code to eliminate deeply nested if-elif-else
structures. Improve readability, simplify logic, and ensure the code is more
maintainable. Use guard clauses or a mapping/dictionary-based approach
instead of nested conditionals.
Legacy Code:
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
Expected:
- Flattened structure (no deep nesting)
- Cleaner logic
- Easy to extend and test'''
def evaluate_score(score):
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"
# Example usage
if __name__ == "__main__":
    score = float(input("Enter the score: "))
    print(evaluate_score(score))
```

Output:

```
PS C:\Users\keert\auth-system> python -u "c:\Users\keert\OneDrive\Docume
Enter the score: 99
Excellent
PS C:\Users\keert\auth-system> python -u "c:\Users\keert\OneDrive\Docume
Enter the score: 21
Needs Improvement
PS C:\Users\keert\auth-system> python -u "c:\Users\keert\OneDrive\Docume
Enter the score: 78
Very Good
PS C:\Users\keert\auth-system> python -u "c:\Users\keert\OneDrive\Docume
Enter the score: 65
Good
PS C:\Users\keert\auth-system>
```

Observation:
The refactored code removes deep nesting and uses a cleaner, flatter structure, which improves readability. Logical flow becomes straightforward, making the program easier to maintain and extend. Test results show consistent and accurate output for different score inputs, proving that the simplified structure works reliably.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

```
f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.

Prompt:

Refactor the repeated file handling code using Python.Apply DRY principle and context managers (with open()). Create a reusable function that accepts the filename as a parameter, removes duplicate open/read/close code, improves variable naming, and adds comments explaining the logic.Take input from the user for the filename.

Code:

```python
'''
Refactor the repeated file handling code using Python.
Apply DRY principle and context managers (with open()).
Create a reusable function that accepts the filename as a parameter, removes duplicate open/read/close code,
improves variable naming, and adds comments explaining the logic.Take input from the user for the filename.'''
def read_and_print_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
    except IOError:
        print(f"An error occurred while trying to read the file '{filename}'.")
# Take input from the user for the filename
filename = input("Enter the filename to read: ")
read_and_print_file(filename)
```

Output:

```
Enter the filename to read: data.txt
Error: The file 'data.txt' was not found.
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\keert\OneD
Enter the filename to read: lab-3.2.py
#generate a python script that logs user activity including username, IP address, and
import logging
from datetime import datetime
# Configure logging to write a file with the specific format
logging.basicConfig(filename='user_activity.log', level=logging.INFO, format='%(asctim
def log_user_activity(username, ip_address):
    timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    logging.info(f'Username: {username}, IP Address: {ip_address}, Timestamp: {timesta
#example usage
```

Observation:

The updated code removes repeated file-handling statements by introducing a reusable function in which the file name is passed as a parameter. The use of the with open() context manager ensures efficient resource management because the file is automatically closed after the required operations are completed. This approach follows the **DRY (Don't Repeat Yourself) principle**, improves code readability, and makes the program easier to maintain and understand.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt:

Refactor the given Python code that performs a linear search on a list of usernames.

Replace the list-based linear search with a more efficient data structure like a set or dictionary to achieve better time complexity. Include meaningful variable names, comments explaining the changes, and mention the time complexity of the new approach and Take input from the user for the username.

Code:

```python
'''
Refactor the given Python code that performs a linear search on a list of usernames.
Replace the list-based linear search with a more efficient data structure like a set or
dictionary to achieve better time complexity. Include meaningful variable names, comments
explaining the changes, and mention the time complexity of the new approach and Take input
from the user for the username.'''
def check_user_access(username, user_set):
    return "Access Granted" if username in user_set else "Access Denied"
#create a set of valid usernames for efficient lookup
valid_users = {"admin", "guest", "editor", "viewer"}
# Take input from the user for the username
username = input("Enter the username to check access: ")
access_status = check_user_access(username, valid_users)
print(access_status)
```

Output:

```
Enter the username to check access: admin
Access Granted
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab>
                python -u "c:\Users\keert\OneDrive\Documents\AI Assis
Enter the username to check access: admin

ssistance\lab> python -u "c:\Users\keert\OneDrive\Documents\AI Assis
Enter the username to check access: Keerthi
Access Denied
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c
```

Observation:

The updated program improves the efficiency of the search operation by using a **set** instead of a **list**. This change reduces the lookup time from **O(n)** to approximately **O(1)**. The control flow is also simplified by directly checking for membership rather than using a loop with a flag variable. As a result, the code becomes cleaner, easier to understand, and more efficient in terms of performance.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt:

Refactor the given Python code into an Object-Oriented design. Create a class named EmployeeSalaryCalculator that follows OOP principles and encapsulation. Include attributes for salary and tax_rate, and methods to calculate tax and net salary. Adding proper documentation. Ensure the code is clean, modular, and readabile. Take input from the user for salary and tax rate.

Code:

```python
'''
Refactor the given Python code into an Object-Oriented design. Create a class named
EmployeeSalaryCalculator that follows OOP principles and encapsulation. Include attributes for salary and tax_rate,
and methods to calculate tax and net salary. Adding proper documentation. Ensure the code is clean, modular, and
readabile. Take input from the user for salary and tax rate.'''
class EmployeeSalaryCalculator:
    """
    A class to calculate the tax and net salary of an employee based on their gross salary and tax rate.
    """

    def __init__(self, salary: float, tax_rate: float):
        """
        Initializes the EmployeeSalaryCalculator with the given salary and tax rate.

        :param salary: The gross salary of the employee.
        :param tax_rate: The tax rate to be applied (as a percentage).
        """
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self) -> float:
        """
        Calculates the tax amount based on the salary and tax rate.

        :return: The calculated tax amount.
        """
        return self.salary * (self.tax_rate / 100)

    def calculate_net_salary(self) -> float:
        """
        Calculates the net salary after deducting the tax from the gross salary.

        :return: The calculated net salary.
        """
        return self.salary - self.calculate_tax()
# Take input from the user for salary and tax rate
try:
    salary = float(input("Enter the gross salary: "))
    tax_rate = float(input("Enter the tax rate (in percentage): "))
    # Create an instance of EmployeeSalaryCalculator
    calculator = EmployeeSalaryCalculator(salary, tax_rate)
    net_salary = calculator.calculate_net_salary()
    print(f"Net Salary: {net_salary:.2f}")
except ValueError:
    print("Invalid input. Please enter numeric values for salary and tax rate.")
```

Output:

```
                python -u "c:\Users\keert\OneDrive\Documents\AI Assistance\lab\
Enter the gross salary: 20000
Enter the tax rate (in percentage): 10
Net Salary: 18000.00
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\ke
Enter the gross salary: 20000
Enter the tax rate (in percentage): 10
Net Salary: 18000.00
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\ke
Net Salary: 18000.00
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\ke
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\ke
Enter the gross salary: 50000
Enter the tax rate (in percentage): 30
Net Salary: 35000.00
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab>
```

Observation:

The procedural code was converted into a class-based design by applying object-oriented programming principles. Encapsulation was implemented by defining salary and tax rate as class attributes and using class methods to perform the required calculations. The improved program structure divides the code into smaller, organized components, which enhances code reusability and makes it easier to maintain or extend with additional features in the future.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)
```

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:

Refactor the following Python code to improve performance and using built-in functions or mathematical formulas.

Rename variables meaningfully and improve the code structure. Compare execution time between the legacy loop implementation and the optimized version. Take input from the user for the limit.

Code:

```python
'''
Refactor the given Python code into an Object-Oriented design. Create a class named
EmployeeSalaryCalculator that follows OOP principles and encapsulation. Include attributes for salary and tax_rate,
and methods to calculate tax and net salary. Adding proper documentation. Ensure the code is clean, modular, and
readabile. Take input from the user for salary and tax rate.'''
import time
def sum_of_even_numbers(limit):
    """
    Calculate the sum of even numbers up to a given limit using a mathematical formula.

    :param limit: The upper limit up to which even numbers are summed.
    :return: The sum of even numbers up to the limit.
    """
    # Using the formula for the sum of the first n even numbers: n * (n + 1)
    n = limit // 2  # Number of even numbers up to the limit
    return n * (n + 1)
# Take input from the user for the limit
try:
    limit = int(input("Enter the upper limit to sum even numbers: "))
    start_time = time.time()
    result = sum_of_even_numbers(limit)
    end_time = time.time()
    print(f"Sum of even numbers up to {limit}: {result}")
    print(f"Execution time: {end_time - start_time:.6f} seconds")
except ValueError:
    print("Invalid input. Please enter an integer value for the limit.")
```

Output:

```
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\keert\
Enter the upper limit to sum even numbers: 0
Sum of even numbers up to 0: 0
Execution time: 0.000007 seconds
PS C:\Users\keepython -u "c:\Users\keert\OneDrive\Documents\AI Assistance\lab\13.2
Enter the upper limit to sum even numbers: 10
Sum of even numbers up to 10: 30
Execution time: 0.000009 seconds
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab>
```

Observation:

The refactored code improves performance by replacing the loop with a direct mathematical formula and using built-in functions. This reduces the execution time and eliminates unnecessary conditional checks inside the loop. As a result, the improved version runs more efficiently while producing the same results, and the code becomes simpler, easier to read, and easier to maintain.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

data = []

def add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values instead of mutating

globals.

Prompt:

Refactor the given Python code to remove hidden side effects caused by modifying a global mutable list. Rewrite the function in a functional style so it returns a new list instead of mutating global state. Add clear comments and Take input from the user.

Code:

```python
'''
Refactor the given Python code to remove hidden side effects caused by modifying a global mutable list.
Rewrite the function in a functional style so it returns a new list instead of mutating global state.
Add clear comments and Take input from the user.'''
def add_item_to_list(item, data_list):
    """
    Add an item to a list and return a new list without modifying the original.

    :param item: The item to be added to the list.
    :param data_list: The original list to which the item will be added.
    :return: A new list containing all items from the original list plus the new item.
    """
    # Create a new list by concatenating the original list with the new item
    return data_list + [item]
# Take input from the user for the item and the list
try:
    item = input("Enter the item to add to the list: ")
    original_list_input = input("Enter the original list items separated by commas: ")
    original_list = [x.strip() for x in original_list_input.split(',')]
    new_list = add_item_to_list(item, original_list)
    print(f"Original List: {original_list}")
    print(f"New List: {new_list}")
except Exception as e:
    print(f"An error occurred: {e}")
```

Output:

```
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\keert\O
Enter the item to add to the list: 26,70
Enter the original list items separated by commas: []
Original List: ['[]']
\Documents\AI Assistance\lab\13.2.py"
Enter the item to add to the list: 12,34
Enter the original list items separated by commas: 34,56
Original List: ['34', '56']
New List: ['34', '56', '12,34']
```

Observation:

The rewritten code eliminates hidden side effects by avoiding modifications to the global list. Instead, the function follows a functional programming approach by creating and returning a new list rather than altering shared mutable data. This makes the program more predictable, easier to maintain, and also simplifies the testing process.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

Prompt:

Refactor the following Python password validation code to improve readability.

Create separate functions: check_length(password), check_digit(password), and check_uppercase(password) that return True/False. Implement a validate_password(password) function that calls these functions and prints appropriate error messages if a rule fails. Add comments and Take input from the user.

Code:

```python
'''
Refactor the following Python password validation code to improve readability.
Create separate functions: check_length(password), check_digit(password), and check_uppercase(password)
that return True/False. Implement a validate_password(password) function that calls these functions and prints
appropriate error messages if a rule fails. Add comments and Take input from the user.'''
def check_length(password):
    """Check if the password is at least 8 characters long."""
    return len(password) >= 8
def check_digit(password):
    """Check if the password contains at least one digit."""
    return any(char.isdigit() for char in password)
def check_uppercase(password):
    """Check if the password contains at least one uppercase letter."""
    return any(char.isupper() for char in password)
def validate_password(password):
    """Validate the password against all rules and print appropriate messages."""
    if not check_length(password):
        print("Password must be at least 8 characters long.")
        return False
    if not check_digit(password):
        print("Password must contain at least one digit.")
        return False
    if not check_uppercase(password):
        print("Password must contain at least one uppercase letter.")
        return False
    print("Password is valid.")
    return True
# Take input from the user for the password
password = input("Enter a password to validate: ")
validate_password(password)
```

Output:

```
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\kee
Enter a password to validate: abcdefg
Password must be at least 8 characters long.
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\kee
Enter a password to validate: abcdefghi
Password must contain at least one uppercase letter.
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\kee
Enter a password to validate: Abc@12345
Password must contain at least one uppercase letter.
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\kee
Enter a password to validate: Abc@12345
Password is valid.
PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab>

PS C:\Users\keert\OneDrive\Documents\AI Assistance\lab> python -u "c:\Users\kee
Enter a password to validate: Abc@12345
Enter a password to validate: Abc@12345
Password is valid.
```

Observation:

The refactored code improves readability by separating each validation rule into its own function. This modular structure makes the program easier to test, debug, and maintain. It also reduces the need for multiple conditional statements, making the code clearer and easier to understand. Additionally, the validation logic can now be reused in different parts of the program. Although the validation rules remain the same as in the original code, the refactored version presents them in a simpler and more organized manner.